UNIVERSIDAD TÉCNICA DE MANABÍ

FACULTAD DE CIENCIAS INFORMÁTICAS

TECNOLOGÍAS DE LA INFORMACIÓN

ACTIVIDAD #1

Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

ESTUDIANTE

Pita Mera Kleber Felipe

DOCENTE

Ing. Jorge Arun Zambrano Cedeño, Mg.

CURSO

Desarrollo De Sistemas Informáticos - “A”

Preparación de GitHub a través de GitBash

```
UCRT64:/e/Users/Kleber Pita/Desktop/Universidad 2/5to semestre/Desarrollo de Sistemas In

Kleber Pita@KLEBER-PITA-LAPTOP UCRT64 /e/Users/Kleber Pita/Desktop/Univer
ollo de Sistemas Informáticos/Actividad_1 (main)
$ git remote add origin https://github.com/kleberpitamera/Act_1_DSI.git

Kleber Pita@KLEBER-PITA-LAPTOP UCRT64 /e/Users/Kleber Pita/Desktop/Univer
ollo de Sistemas Informáticos/Actividad_1 (main)
$ git branch -M main
```

```
Kleber Pita@KLEBER-PITA-LAPTOP UCRT64 /e/Users/Kleber Pita/Desktop/Universic
$ git add .

Kleber Pita@KLEBER-PITA-LAPTOP UCRT64 /e/Users/Kleber Pita/Desktop/Universic
$ git commit -m "Creacion de .txt para enlazar el repositorio"
[main (root-commit) 6e2031d] Creacion de .txt para enlazar el repositorio
 1 file changed, 1 insertion(+)
 create mode 100644 enlazar_repositorio.txt

Kleber Pita@KLEBER-PITA-LAPTOP UCRT64 /e/Users/Kleber Pita/Desktop/Universic
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 280 bytes | 280.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kleberpitamera/Act_1_DSI.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

Kleber Pita@KLEBER-PITA-LAPTOP UCRT64 /e/Users/Kleber Pita/Desktop/Universic
$ git checkout -b develop
Switched to a new branch 'develop'

Kleber Pita@KLEBER-PITA-LAPTOP UCRT64 /e/Users/Kleber Pita/Desktop/Universic
$ git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:      https://github.com/kleberpitamera/Act_1_DSI/pull/new/develop
remote:
To https://github.com/kleberpitamera/Act_1_DSI.git
 * [new branch]      develop -> develop
branch 'develop' set up to track 'origin/develop'.
```

Link GitHub: https://github.com/kleberpitamera/Act_1_DSI

Análisis del Problema

Actores Principales

El sistema de gestión de rescate y adopción animal contempla la interacción de tres roles clave con permisos y responsabilidades específicas:

- **Ciudadano / Reportante:** Actor externo que utiliza la plataforma para notificar emergencias, abandonos o avistamientos de animales en situación de vulnerabilidad (perros, gatos o fauna silvestre).
- **Voluntario / Veterinario:** Personal operativo y clínico encargado de atender los reportes asignados, registrar diagnósticos, actualizar el historial clínico de los animales y dar seguimiento a su proceso de recuperación.
- **Administrador del Refugio:** Gestor del sistema con privilegios avanzados para administrar cuentas, asignar casos al personal disponible, supervisar el inventario y aprobar las solicitudes de adopción final.

Requerimientos Funcionales (RF)

El software debe cumplir con las siguientes capacidades operativas:

- **RF01 - Gestión de Reportes de Rescate:** Permitir al ciudadano crear un caso de rescate detallando ubicación, descripción y evidencia fotográfica.
- **RF02 - Categorización de Especies:** Clasificar automáticamente el ingreso del animal según su tipo (perro, gato o fauna silvestre) para determinar el protocolo de atención adecuado.
- **RF03 - Asignación de Personal:** Permitir al administrador asignar voluntarios o veterinarios específicos a cada caso registrado.
- **RF04 - Gestión de Estados del Proceso:** Permitir actualizar el ciclo de vida del caso a través de estados definidos: *Rescatado*, *En Tratamiento* y *Adopción*.

Requerimientos No Funcionales (RNF)

- **RNF01 - Escalabilidad:** La arquitectura del sistema debe permitir la incorporación futura de nuevas especies o módulos de donaciones sin afectar el núcleo del software.

- **RNF02 - Mantenibilidad:** Aplicación rigurosa de principios de diseño y patrones de software para facilitar modificaciones y pruebas en el código fuente.
- **RNF03 - Trazabilidad de Versiones:** Control estricto de los cambios en el código y la documentación mediante el modelo de trabajo Gitflow en GitHub.

Justificación

En el desarrollo del sistema de gestión y rescate animal para el refugio, se han seleccionado e implementado tres patrones de diseño fundamentales del catálogo de GoF (Gang of Four): Factory Method, Singleton y Observer. La inclusión de estos patrones responde a la necesidad de garantizar un software mantenible, escalable, desacoplado y orientado al cumplimiento de los principios SOLID.

Patrón Creacional: Factory Method (Fábrica de Animales)

En el refugio ingresan continuamente distintos tipos de especies (`Perro`, `Gato`, `FaunaSilvestre`), cada una con propiedades y requerimientos de cuidado específicos. Si la lógica de instanciación se realizara mediante llamadas directas al operador `new` en múltiples partes de la aplicación, el sistema quedaría fuertemente acoplado a las clases concretas. Esto violaría el Principio de Abierto/Cerrado (OCP), ya que añadir una nueva especie en el futuro requeriría modificar el código existente en toda la aplicación.

Solución: Se implementó el patrón Factory Method a través de la clase `AnimalFactory`, la cual centraliza la responsabilidad de instanciar los objetos derivados de la clase abstracta `Animal`.

- **Estructura en el diseño:**
  - **Clase Abstracta:** `Animal` (define el contrato general y atributos comunes).
  - **Productos Concretos:** `Perro`, `Gato` y `FaunaSilvestre`.
  - **Creador/Fábrica:** `AnimalFactory`, que expone el método `+ crearAnimal(tipoEspecie: String): Animal`.
- **Impacto en el Software:**
  - Desacopla la lógica de negocio de la instanciación directa de clases.

- Permite incorporar nuevas especies de animales (ej. *Aves*) creando una nueva subclase sin alterar el código del creador ni de los clientes.

Patrón Creacional: Singleton (Gestor Central de Notificaciones)

El envío de alertas y notificaciones ante eventos críticos (como el reporte de un nuevo rescate) debe ser coordinado por una entidad única dentro de la aplicación. Si existieran múltiples instancias de la clase encargada de las notificaciones, se generarían inconsistencias en el estado del sistema, consumo innecesario de recursos de memoria y riesgo de duplicación en el envío de mensajes a los voluntarios.

Solución: Se aplicó el patrón Singleton en la clase `NotificacionManager`. Este patrón asegura que exista una sola instancia activa de la clase durante todo el ciclo de vida de la aplicación y proporciona un punto de acceso global a ella.

- **Estructura en el diseño:**
  - **Atributo Privado Estático:** `-_instancia: NotificacionManager`, que almacena la única referencia existente.
  - **Constructor Privado:** Impide que otras clases instancien la clase directamente mediante el operador `new`.
  - **Método de Acceso Público Estático:** `+ getInstancia(): NotificacionManager`, que retorna la instancia única (creándola de manera perezosa si no existe).
- **Impacto en el Software:**
  - Garantiza la consistencia global en la emisión de alertas.
  - Optimiza el uso de recursos al evitar instancias innecesarias.

Patrón Comportamental: Observer (Suscripción y Notificación de Voluntarios)

Cuando un ciudadano registra un nuevo rescate, los voluntarios y personal veterinario deben ser informados de manera inmediata para atender el caso. Utilizar un mecanismo de consulta continua (*polling*) por parte de los voluntarios resulta ineficiente y satura el sistema. Se requiere un mecanismo de comunicación uno-a-muchos basado en eventos de manera totalmente asíncrona y desacoplada.

Solución: Se combinó el patrón Observer dentro de la estructura de `NotificacionManager` (actuando como el *Sujeto/Observable*) para gestionar la suscripción y notificación a las instancias de `VoluntarioVeterinario` (quienes actúan como los *Observadores*).

- **Estructura en el diseño:**
  - **Sujeto (*Subject*):** `NotificacionManager` mantiene una lista privada de suscriptores (`- observadores: List<VoluntarioVeterinario>`).
  - **Métodos de Gestión:** `+ suscribir(voluntario: VoluntarioVeterinario)` para registrar nuevos observadores y `+ notificarVoluntarios(mensaje: String)` para iterar sobre la lista y emitir la alerta automáticamente a cada uno.
  - **Observadores (*Observers*):** La clase `VoluntarioVeterinario` recibe y procesa las alertas emitidas por el gestor.
- **Impacto en el Software:**
  - **Fomenta el Bajo Acoplamiento**: `NotificacionManager` no necesita conocer los detalles internos de los voluntarios, solo mantiene su referencia para notificarles.
  - Comunicación reactiva en tiempo real ante la creación de nuevos casos de rescate.

Diagrama Caso de Uso

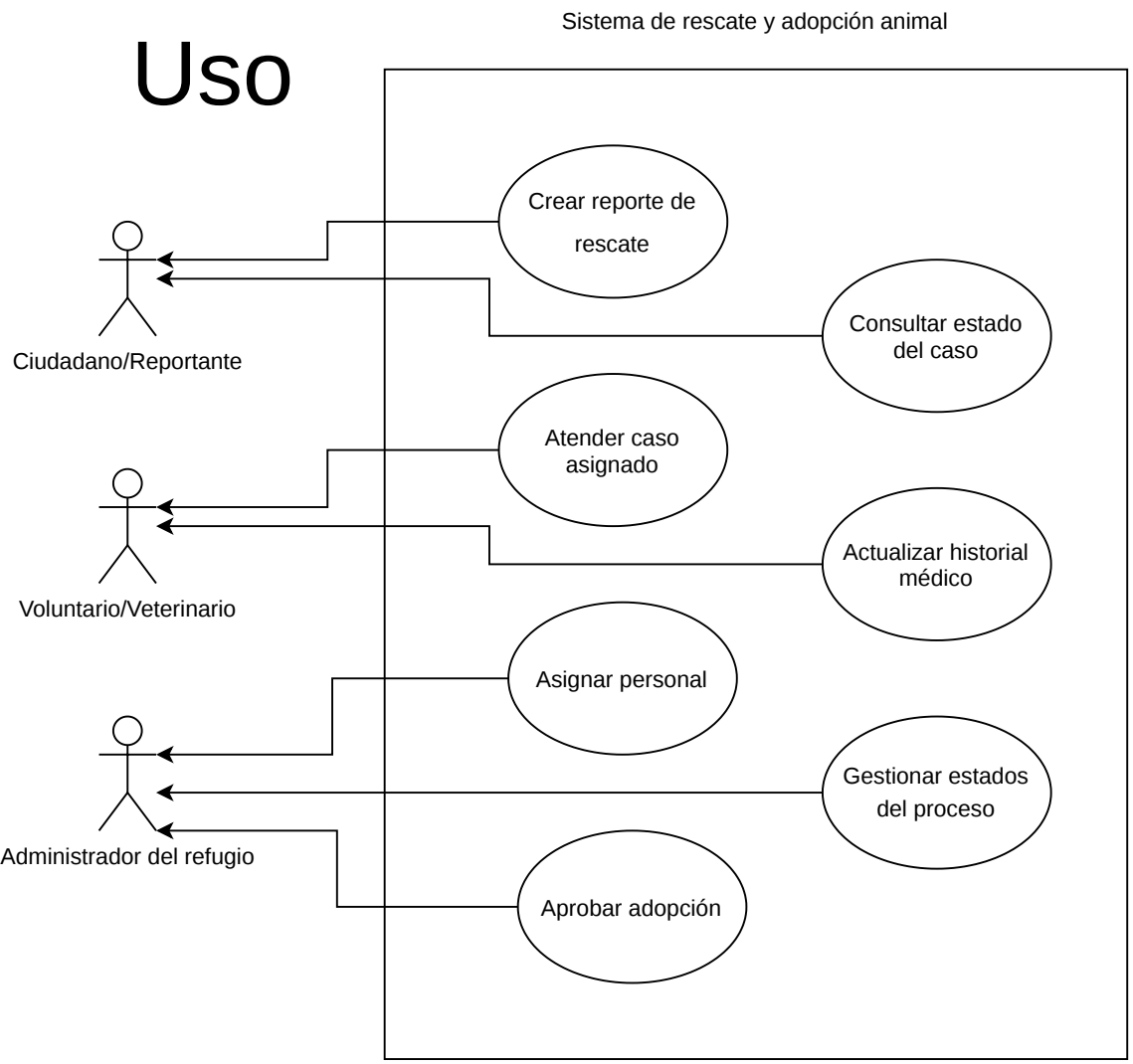


Diagrama UML

